PRODUCT & ENGINEERING BLUEPRINT

Tennis Prediction App Complete Development Roadmap

A phased plan for building a live tennis data platform that runs a custom Abjad-based calculation, classifies remainder patterns, compares historical matches, and produces evidence-based prediction strength.

Core principle: build the calculation and historical evidence engine first. The mobile interface comes after the backend can ingest live tennis data, produce reproducible calculations, freeze predictions, and verify results automatically.

Prepared from the agreed project design - October 2026

1. Freeze the Calculation Rules

Before building the user interface, create a formal written specification for the formula. For every tennis match, the engine must know:

- Player 1 name and Player 2 name
- Arabic semantic equivalent for each name
- Abjad value for each Arabic name
- Country and city
- Local weekday, Gregorian date, and local start time
- Hijri month and Hijri year
- Constant base value: 4064
- Environmental base total
- Player totals and modulo-9 remainders

```
Example
Anton
Arabic calculation name: NAFIS
Name value: 200

Tamworth: 1147
Australia: 703
Thursday: 741
Jumada al-Awwal: 136
Hijri year: 1448
Constant: 4064

Environmental base:
4064 + 703 + 1147 + 741 + 136 + 1448 = 8239

Anton:
8239 + 200 = 8439
8439 mod 9 = 6
```

Milestone 1: manually enter one match and get the correct two remainders every time. Do not build prediction logic until this is reliable.

2. Build the Master Name Dictionary

The app should not reinterpret the same player's name every time. Once a semantic Arabic mapping is approved, it should be stored and reused consistently.

Field	Example
original_name	Anton
normalized_name	anton
arabic_name	NAFIS (Arabic semantic mapping)
semantic_meaning	precious / valuable
abjad_value	200
mapping_type	semantic

Field	Example
approved	true
version	1

```
New player detected
        |
        v
Extract given name
        |
        v
Check name database
   |            |
 found       not found
   |            |
 reuse       propose Arabic semantic mapping
                |
                v
          admin approval
                |
                v
          save permanently
```

At launch, unfamiliar mappings should be manually reviewed. As the dictionary grows, most calculations will become automatic.

3. Connect a Live Tennis Data Provider

The tennis data layer should provide, where available:

• Upcoming matches

• Player names and IDs

• Tournament and competition level

• Scheduled start time

• Match status and live score

• Completed result and winner

• Tournament city/country or enough metadata to resolve the venue

```
Tennis API
    |
    v
Fetch today + tomorrow
    |
    v
Store raw provider data
    |
    v
Normalize players / tournament / venue / timezone
    |
    v
Calculate local date and time
    |
    v
Run formula
    |
    v
```

```
Store remainder pair
```

> Important: keep API credentials and proprietary calculation logic on the server. The mobile app should request processed data from your backend rather than calling the tennis provider directly.

4. Design the Database Correctly

PostgreSQL is a strong fit; Supabase can provide PostgreSQL, authentication, realtime updates, storage, and server functions.

players

```
id
provider_player_id
full_name
first_name
country
name_mapping_id
created_at
```

locations

```
id
country
country_arabic
country_abjad
city
city_arabic
city_abjad
timezone
latitude
longitude
```

matches

```
id
provider_match_id
player1_id
player2_id
tournament_id
location_id
scheduled_time_utc
scheduled_time_local
actual_start_time_utc
actual_start_time_local
status
winner_id
created_at
updated_at
```

calculations

```
match_id
constant
country_value
city_value
weekday_value
hijri_month_value
hijri_year
environment_base
player1_name_value
player2_name_value
player1_total
player2_total
```

```
player1_remainder
player2_remainder
formula_version
calculated_at
```

5. Store Structural Remainder Features

The app should not only store the exact pair. Every match should also be classified by parity and relative size.

```
Example: 3 vs 8

remainder_a = 3
remainder_b = 8
a_parity = odd
b_parity = even
a_relative_size = smaller
b_relative_size = bigger
pair_class = smaller_odd_vs_bigger_even
parity_class = odd_vs_even
magnitude_class = smaller_vs_bigger
```

If 3 wins, also store:

```
winning_remainder = 3
winning_parity = odd
winning_size = smaller
winning_class = smaller_odd
```

Example	Structural interpretation
3 > 5	Smaller odd beats bigger odd
5 > 3	Bigger odd beats smaller odd
2 > 8	Smaller even beats bigger even
8 > 2	Bigger even beats smaller even
3 > 8	Smaller odd beats bigger even
8 > 3	Bigger even beats smaller odd
7 > 2	Bigger odd beats smaller even
2 > 7	Smaller even beats bigger odd

6. Build the Historical Pattern Engine

When a new match produces, for example, 3 vs 5, the engine should search several evidence layers instead of only one.

A. Exact pair

```
3 vs 5
Historical record: 3 won 7, 5 won 2
```

B. Structural class

```
smaller odd vs bigger odd
```

C. Parity class

```
odd vs odd
```

D. Magnitude class

```
smaller vs bigger
```

E. Environment

- Same city
- Same country
- Same weekday
- Same Hijri month
- Same Hijri date, if later shown to be useful
- Time proximity
- Same tournament, if useful
- Maximum 12-hour comparison window

Key principle: exact remainder evidence is the most specific, but structural patterns give the system useful evidence even when an exact pair has little or no history.

7. Create an Environmental Similarity Score

Rather than treating every historical match equally, assign a similarity score. The numbers below are placeholders and should later be calibrated from real data.

Condition	Illustrative weight
Exact remainder pair	+30
Same structural class	+15
Exact city	+20
Same country	+5
Same weekday	+10
Same Hijri month	+5
Time difference <= 2 hours	+15
Time difference 2-4 hours	+10
Time difference 4-8 hours	+5
Time difference 8-12 hours	+2

Do not assume these weights are correct. Use them only as an initial prototype. Once enough historical predictions exist, analyze which variables actually improve predictive performance.

8. Build the Prediction-Strength Engine

Prediction strength should combine four major factors:

1. Historical consistency - how often the same side/category wins.

2. Sample size - 1/1 is not stronger evidence than 17/20.

3. Environmental similarity - highly similar matches should carry more weight.

4. Agreement across layers - exact pair, structural class, location, weekday, and time should ideally point in the same direction.

```
Example of strong agreement

Exact pair:
3 beats 5 -> 8/9

Smaller odd vs bigger odd:
smaller odd wins -> 31/40

Same city:
smaller odd wins -> 12/14

Same weekday/time:
smaller odd wins -> 9/10

Result: multiple evidence layers agree
```

When layers disagree, prediction strength should drop even if one individual statistic looks strong.

9. Use Prediction Classes Before Percentages

Class	Meaning
VERY STRONG	Large enough sample, highly similar matches, strong agreement, little contradictory evidence.
STRONG	Good historical agreement and environmental similarity.
MODERATE	Useful evidence, but limited sample or mild contradiction.
WEAK	Small samples, distant matches, or uncertain evidence.
NO PREDICTION	Evidence is too limited or meaningfully contradictory.

After thousands of predictions, map these labels to measured historical accuracy. Only then should a label be translated into a displayed probability.

10. Define the Match Lifecycle

```
UPCOMING
   |
   v
CALCULATED
   |
   v
PREDICTION AVAILABLE
   |
   v
LIVE
   |
   v
COMPLETED
   |
   v
RESULT VERIFIED
   |
   v
ADDED TO HISTORICAL DATA
```

The most important rule is to freeze the prediction before the outcome is known.

```
prediction_created_at
predicted_side
prediction_strength
evidence_available_at_prediction_time
prediction_engine_version
```

Never rewrite a historical prediction after the match finishes.

11. Handle Scheduled Time vs Actual Start Time

Tennis schedules move. A match scheduled for 2:00 PM may actually begin at 3:17 PM. Since your method uses time, store both values:

```
scheduled_start_time
actual_start_time
```

A useful design is to preserve two calculations:

```
pre_match_calculation
live_start_calculation
```

This will later let you test whether scheduled time or actual start time produces better historical performance.

12. Build an Internal Admin Dashboard First

Before releasing a public app, create an internal dashboard that lets you inspect calculations, mappings, and evidence.

```
TODAY'S MATCHES

M25 Tamworth
Anton Shepp vs Isaac Becroft

Local time: 14:08
Country: Australia
City: Tamworth

Name mapping:
```

```
Anton -> approved semantic mapping -> 200
Isaac -> approved semantic mapping -> 170

Calculation:
6 vs 3

Pattern:
Bigger even vs smaller odd

Historical exact pair:
6 vs 3 -> 4-1

Structural history:
Bigger even vs smaller odd -> 18-7

Environment:
Same city: 4 matches
Same weekday: 11 matches
Within 4 hours: 7 matches

Prediction: Player A
Strength: STRONG
```

This dashboard is where calculation errors, bad location data, inconsistent name mappings, and timing problems should be caught before users see them.

13. Build the Mobile App

Recommended front end: React Native + Expo + TypeScript, allowing one codebase for iOS and Android.

Home

```
LIVE | UPCOMING | COMPLETED
```

Prediction filters

```
VERY STRONG | STRONG | MODERATE | ALL
```

Match card

```
M25 Tamworth
Anton Shepp vs Isaac Becroft
6 vs 3
Prediction: ANTON SHEPP
Strength: STRONG
Starts: 2:15 PM local
```

Match detail

- Full calculation
- Historical exact-pair evidence
- Structural pattern evidence
- Location and time evidence
- Previous comparable matches
- Live score
- Final result

History

```
Last 30 days
VERY STRONG: 41 predictions, 32 correct, 9 incorrect
```

```
STRONG: 93 predictions, 66 correct, 27 incorrect
```

14. Build the Results Feedback Loop

```
Provider supplies winner
        |
        v
Database updates match
        |
        v
Compare to frozen prediction
        |
        v
Correct / Incorrect
        |
        v
Update historical statistics
        |
        v
Match becomes evidence for future matches
```

The engine improves because its historical evidence grows, not because the app silently changes old predictions.

15. Let the Data Determine the Weights

At first, you may believe exact pair, city, weekday, time, and Hijri month are all important. After several thousand matches, test each one statistically.

Variable	Possible observed effect
Exact pair	Very high
Structural class	High
City	High
Weekday	Medium
Time distance <= 4 hours	High
Hijri month	Low
Country	Low
Tournament	Medium

The table above is illustrative only. The actual effects must be measured from your own dataset.

16. Add Formula and Version Control

Every calculation and prediction should record which rules created it.

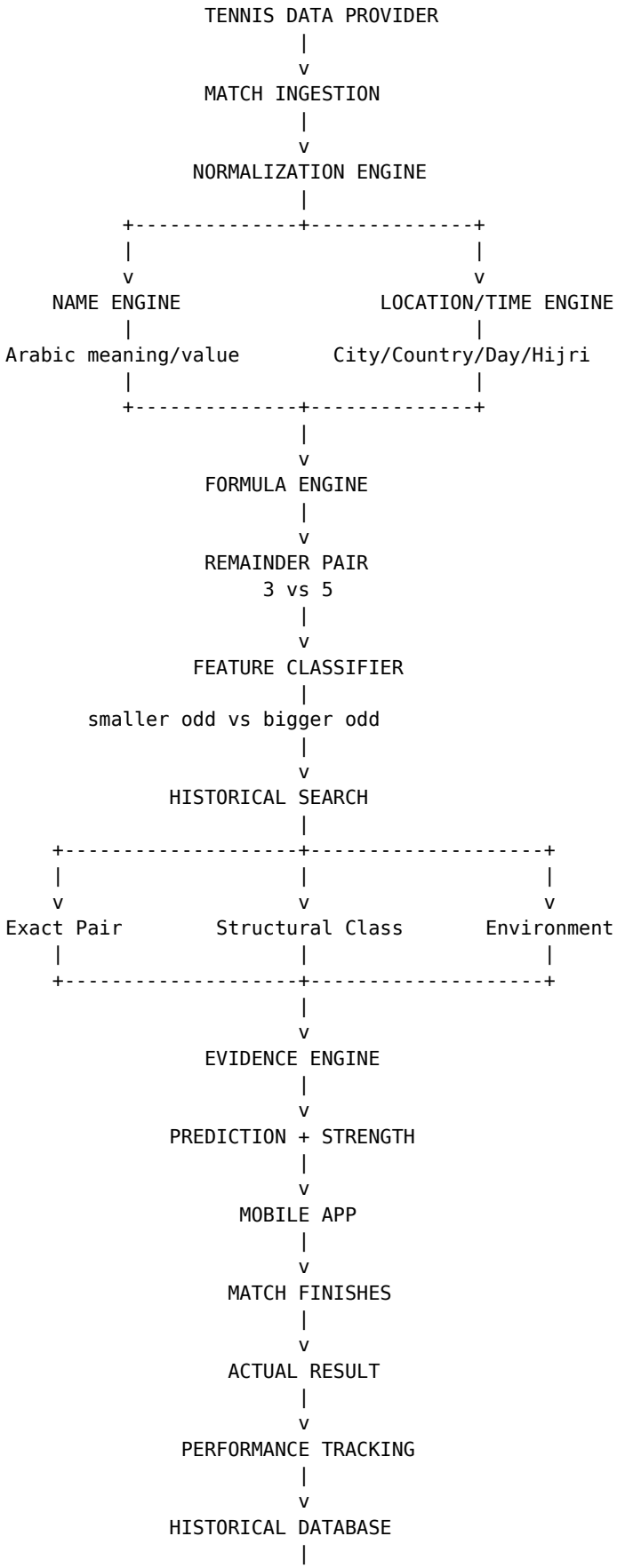
```
formula_version = 1.0
prediction_engine_version = 1.0
name_dictionary_version = 4
```

If you later change the maximum time window from 12 hours to 6 hours, create a new prediction-engine version rather than rewriting old data. That allows clean comparison between rule versions.

17. Recommended Technical Stack

Layer	Recommended technology
Mobile	React Native, Expo, TypeScript
Backend	Node.js / TypeScript or Supabase Edge Functions
Database	PostgreSQL / Supabase
Background workers	Match importer, calculation worker, prediction worker, result updater
External data	Tennis data API, timezone data, Hijri calendar conversion, name-resolution workflow
Realtime delivery	Supabase Realtime or WebSockets

18. Complete System Architecture



```
                                  +----> Future predictions
```

19. Recommended Build Order

1. Manual calculation engine: enter one match and reproduce the correct remainder pair.
2. Name database: lock approved Arabic mappings and Abjad values.
3. Upcoming match ingestion: automatically pull scheduled tennis matches.
4. Automatic calculations: run the formula for every incoming match.
5. Automatic result capture: collect completed winners and actual start times.
6. Historical import: enter the results already gathered manually.
7. Exact-pair matching: compare remainder pairs such as 3 vs 5.
8. Structural matching: add odd/even and bigger/smaller categories.
9. Environmental matching: add city, weekday, Hijri month/date, and time-distance logic.
10. Prediction strength: add sample-size, agreement, contradiction, and similarity scoring.
11. Admin dashboard: inspect calculations and backtest before public release.
12. Mobile app: live, upcoming, completed, prediction detail, and history screens.
13. Performance measurement: freeze predictions and track real-world accuracy.
14. Weight calibration: tune the evidence model using accumulated data.

20. Practical MVP Definition

The first usable version does not need every final feature. A sensible MVP should:

- Pull upcoming matches for selected competitions.
- Resolve player name mappings from the approved dictionary.
- Calculate each match automatically.
- Store the two remainders and their structural class.
- Search historical exact-pair and structural-category evidence.
- Restrict comparison to appropriate city/day/time conditions.
- Produce a strength label or NO PREDICTION.
- Freeze the prediction before the result.
- Update the match automatically when the final winner is available.
- Add the completed match to the historical evidence base.

Best development strategy: prove the calculation and evidence engine with an internal dashboard first. Once that is stable, the public mobile app becomes much easier to build and maintain.

21. Immediate Next Development Tasks

1. Write the formula specification as code-independent rules.
2. Define the exact database schema.
3. Choose the first tennis data provider and test coverage for the tournaments you care about.

4. Import the existing Australia, Singapore, Brazil, Japan, India, and United States results into structured records.

5. Create a prototype calculation API that accepts a match and returns both remainder values plus classification.

6. Create the historical search function for exact pair and structural class.

7. Add similarity weighting by city, weekday, Hijri month/date, and time distance.

8. Build the internal dashboard.

9. Backtest before releasing predictions publicly.

This roadmap is a product and engineering plan. Prediction labels should be validated against a sufficiently large out-of-sample dataset before being presented as measured probabilities.